qual_summary.py 사용 설명서

1. 개요

src/qual_summary.py 는 FASTQ 파일을 읽어서 **모든 read 의 평균 품질 점수(Phred score)** 를 계산해 한 줄로 출력하는 스크립트이다.

- 각 read 마다 품질 문자열을 숫자 점수로 바꾼 뒤 평균을 낸다.
- 그 read 별 평균들을 다시 평균 내어 "전체 read 평균 품질" 로 출력한다.

표준 라이브러리(sys)만 사용하므로 별도 설치 없이 실행된다.

2. 사용법

lab11 폴더 안에서 실행한다.

```
cd lab11
python3 src/qual_summary.py <FASTQ 파일 경로>
```

실행 예시

```
python3 src/qual_summary.py data/SRR1553607_1.fastq
```

출력 결과:

```
전체 read 평균 품질: 33.4497222958299
```

같은 시료의 두 번째 파일(paired-end 의 read 2)도 확인해 보면:

```
python3 src/qual_summary.py data/SRR1553607_2.fastq
전체 read 평균 품질: 30.830840894308746
```

파일	read 개수	read 길이	전체 평균 품질
SRR1553607_1.fastq	203,445	101	약 33.45
SRR1553607_2.fastq	203,445	101	약 30.83

Illumina paired-end 데이터에서는 보통 read 2 의 품질이 read 1 보다 조금 낮다. 위 결과도 그런 경향을 보여 준다.

3. 품질 점수(Phred score) 읽는 법

FASTQ 의 4번째 줄은 서열의 각 염기에 대한 품질을 **문자 하나씩** 으로 적어 둔 것이다.

```
@SRR1553607.1 1 length=101          ← 1줄: 이름
GTTAGCGTTGTTGATCGCGACG...           ← 2줄: 서열
+SRR1553607.1 1 length=101          ← 3줄: 구분선
BB@FFFFFHHHHHJJJJJJJJJ...           ← 4줄: 품질 (서열과 길이가 같다)
```

문자를 점수로 바꾸는 규칙(Phred+33)은 다음과 같다.

```
점수 = ord(문자) - 33
```

ord() 는 문자의 아스키(ASCII) 번호를 돌려주는 파이썬 함수이다.

문자	ASCII 번호	점수 Q	그 염기가 틀렸을 확률
#	35	2	약 63%
+	43	10	10% (1/10)
5	53	20	1% (1/100)
?	63	30	0.1% (1/1,000)
I	73	40	0.01% (1/10,000)
J	74	41	약 0.008%

점수와 오류 확률의 관계는 Q = -10 × log10(오류 확률) 이다. 보통 **Q30 이상**이면 품질이 좋다고 본다. 이 실습 데이터의 품질 문자는 #(Q2) 부터 J(Q41) 까지 나온다.

4. 코드 설명

4.1 read_fastq(filename) 함수

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records
```

parse_fastq.py 의 함수와 똑같다. 파일을 4줄씩 끊어서 read 하나를 (이름, 서열, 품질) 튜플로 만들고, 그 튜플들의 리스트를 돌려준다. 자세한 설명은 doc/parse_fastq.md 의 4.1 절을 참고한다.

4.2 mean_quality(qual) 함수

```
def mean_quality(qual):
    scores = []
    for ch in qual:
        scores.append(ord(ch) - 33)
    return sum(scores) / len(scores)
```

코드	**하는 일**
for ch in qual:	품질 문자열을 한 글자씩 꺼낸다.
ord(ch) - 33	문자를 Phred 점수(숫자)로 바꾼다.
scores.append(...)	점수를 리스트에 모은다.
sum(scores) / len(scores)	점수의 합을 개수로 나눠 read 하나의 평균 품질을 구한다.

예: 품질 문자열이 "BB@F" 이면 점수는 [33, 33, 31, 37] 이고, 평균은 134 / 4 = 33.5 이다.

4.3 메인 부분

```
reads = read_fastq(sys.argv[1])

total = 0
for name, seq, qual in reads:
    q = mean_quality(qual)
    total = total + q

print("전체 read 평균 품질:", total / len(reads))
```

1. sys.argv[1] (명령어 뒤에 적은 파일 경로) 로 FASTQ 파일을 읽는다.
2. read 마다 mean_quality() 로 평균 품질을 구해 total 에 더한다.
3. total 을 read 개수로 나눠 **read 별 평균 품질의 평균** 을 출력한다.

5. 주의할 점과 한계

상황	**결과**
파일 이름을 주지 않고 실행함	sys.argv[1] 이 없어서 IndexError 가 난다.
빈 파일	reads 가 빈 리스트라서 마지막 줄의 total / len(reads) 에서 ZeroDivisionError 가 난다.

품질 줄이 비어 있는 read 가 있음	mean_quality() 의 len(scores) 가 0 이라서 ZeroDivisionError 가 난다.
파일 줄 수가 4의 배수가 아님 (파일이 잘렸거나 끝에 빈 줄이 있음)	read_fastq() 에서 IndexError 가 난다.
read 길이가 서로 다름 (예: fastp 로 트리밍한 뒤)	이 스크립트는 "read 별 평균의 평균" 을 낸다. 짧은 read 와 긴 read 가 같은 비중으로 계산되므로, "모든 염기의 평균 품질" 과 값이 달라진다. 이 실습 데이터는 모든 read 가 101 bp 라서 두 값이 같다.
Phred+64 로 기록된 옛날 Illumina 데이터	33 을 빼도록 고정되어 있어서 점수가 31 만큼 높게 나온다.
압축 파일(.fq.gz)	open() 으로는 읽을 수 없다. 먼저 gunzip 으로 풀어야 한다.
매우 큰 파일	readlines() 가 파일 전체를 메모리에 올린다. 이 실습 파일(약 20만 read, 1.5초 정도)은 문제없지만 수천만 read 짜리 파일에서는 메모리가 부족할 수 있다.

6. 더 해 볼 만한 것

- 실행할 때 파일 이름을 안 주면 사용법을 알려 주고 끝내기 (len(sys.argv) 검사)
- 평균뿐 아니라 최소·최대 품질, Q30 이상인 염기의 비율(%) 구하기
- 위치(1번째 염기, 2번째 염기, …)별 평균 품질을 구해 read 끝으로 갈수록 품질이 떨어지는지 확인하기
- 결과를 FastQC(fastqc data/SRR1553607_1.fastq) 의 "Per sequence quality scores" 그림과 비교하기